Emergency Training Simulator POC

Markdown documentation PDF copy

Speaker notes

Source: docs/presentations/project-overview-speaker-notes.md

Markdown stays editable. This PDF is the reading/share copy.

Boundary: this project is still a POC, not operational guidance.

Speaker notes

These notes match project-overview-slides.md.

The tone is meant to be clear, casual, and confident. More campus project presentation than Shark Tank. A little pitch energy is fine, but the main vibe is: "Here is what I built, here is why I built it in this order, and here is what comes next."

1. Emergency Training Simulator POC

Start simple.

Say that this is a proof-of-concept for an AI-driven adaptive emergency-response training simulator. Then immediately make the scope clear: this is not the final system, not clinical guidance, and not operational authority.

The point of this presentation is to show the project path so far.

2. First audience

This slide sets the tone.

Say that the first audience is Dr. Cameron, so the presentation is meant to work as a clear project/job progress update.

The point is not to sell a finished product. The point is to show careful engineering progress: what exists, what it proves, what is still out of scope, and what should come next.

3. The short version

Explain the core idea in normal language.

The full system vision is much bigger than the repo: AI, immersive environments, sensing, validation, deployment, and review.

The current repo is the first foundation step. It takes a trainee action, updates the scenario, checks the result, sends state to a client, and logs the session for review.

This is the first real foundation. Everything else depends on this working cleanly.

4. Full-system north star

This slide connects the POC to the bigger project overview.

Say that the full system eventually includes agentic AI, immersive rooms, sensing, validation, hub-and-spoke deployment, and instructor review.

Then make the boundary clear: this repo does not claim those features yet. It proves the first trusted backend loop that those later layers will need.

5. Why this project exists

Talk about training as a feedback problem.

Training is not only about making a cool simulation. It is about repeatable practice and useful review. If the system cannot explain what happened afterward, it is not very useful

for learning.

6. The problem I am avoiding

This is where you can be a little funny.

It is tempting to start with the shiny stuff: Unreal, AI, 3D models, sensors, physics, all of it. But if the backend is messy, the shiny layer just hides the mess.

So the project starts with a boring but important question: can the system keep the truth straight?

7. How I started

Describe Sprint 0 as setting up the project rails.

The first pass was docs, schemas, scenario files, validation rules, placeholders, and architecture notes. That might sound slow, but it made the next steps safer and cleaner.

8. The POC loop

Walk through the loop slowly.

Manual action comes in. Backend receives it. Engine updates scenario state. Validation checks the state. Client gets an update. Log gets written. Review can explain it later.

This is the heartbeat of the repo.

9. Where I am now

This is the progress slide.

The backend is no longer just docs. It has a FastAPI WebSocket, deterministic scenario engine, validation, basic security guardrails, logging, replay, review, and a local demo command.

Keep the phrasing grounded. It works locally. It proves the loop. It does not claim more than that.

10. Current scenario

Explain that the first scenario is intentionally tiny.

It is a smoke-room style software scenario with a simple happy path. The point is not to teach real response protocol. The point is to exercise the code path.

11. Environment cues

The wooden chair and entry door matter because they make the room state visible to the backend.

The chair can block the path. The door can be closed. These are simple, obvious environmental cues that the system should not ignore.

12. Why the chair and door matter

This is the bridge to future physics and Unreal.

Right now, these objects are metadata and state. Later, they can grow into richer objects with collision, mass, force direction, gravity notes, and display behavior.

The key is that the backend already knows they exist.

13. Validation boundary

Be direct here.

Validation means rule-conformance for this toy scenario. It does not mean official medical or firefighting correctness.

This is important because the project can be technically useful while still being honest about its limits.

14. Security baseline

Explain the security idea in plain language.

The client is allowed to request an action. The backend decides what actually happened.

That means a future Unreal client should not directly edit scenario state, validation results, final outcome, or review logs.

This is the trust boundary.

15. Current guardrails

Keep this practical.

The current backend rejects unexpected fields, weird IDs, path-like scenario names, unsafe action names, oversized payloads, wrong sequence numbers, and duplicate event IDs.

Also say clearly that this is not production security yet. It is just the first reasonable security floor for a local POC.

16. Sample demo result: happy path

Describe what the good run proves.

The trainee follows the expected order. The chair is moved. The door is opened. The final state reaches the end condition. The replay passes. The review is clean.

That is the "green path."

17. Sample demo result: warning/error path

Describe what the bad run proves.

The trainee tries to move too fast or ignores an environmental cue. The backend warns or errors. The state does not advance incorrectly. The log still records the attempt.

This matters because mistakes are part of training.

18. Sample review output

This is one of the strongest slides for Dr. Cameron.

Presenter note: the slide should not sound like private self-talk. Frame it as context for

the audience: this output is the proof point behind the demo.

Say that this is the difference between a toy demo and a useful training foundation.

A toy demo says: "It ran."

This review says: "Here is what happened, in order, with warnings, errors, replay verification, and environment cues."

The key phrase is: the system can explain what happened.

19. Unreal client contract

Make this distinction clear:

Unreal will display and send input. It should not decide scenario truth.

The repo already has a contract and a Python harness that behaves like the future Unreal client, but there is no scene yet.

20. Asset pipeline direction

Explain the asset plan without claiming assets exist.

Tools like Hunyuan 3D may help generate candidates later, but every asset should go through manual Blender review before it becomes part of the Unreal side.

For now, the chair and door are scenario objects, not meshes.

21. Next goal

The next goal is making the display contract stronger.

That means a future Unreal client can show state clearly without owning the logic. Text-only debug display first. 2D UI and 3D visuals later.

22. How I get there

This is the roadmap slide.

Emphasize controlled steps. Keep the backend clean. Keep tightening the security boundary. Add fields carefully. Keep replay and review strong. Only then bring in Unreal, Blender, generated assets, and physics.

23. Ideal end result

Describe the bigger picture.

The dream is an adaptive training simulator where scenarios respond to trainee actions, environmental cues matter, instructors can review sessions, and AI helps create safe training content.

But again: that is the direction, not the current claim.

24. What this is not

This is the honesty slide.

Say what it is not so the audience trusts what it is.

It is not certified, not medical guidance, not a production simulator, not production-secure, and not a real physics engine.

25. Closing

End with the ordering principle:

Prove the loop first. Then replay and review. Then client display. Then security guardrails. Then richer objects. Then Unreal, Blender, generated assets, and physics.

The closer can be: one monster at a time.

26. Best 10-minute demo flow

This is the practical ending if Dr. Cameron asks how you would present it live.

Use 5 minutes for the story and 5 minutes for proof on screen.

The key sentence is:

The backend can keep track of what happened, explain it afterward, and leave room for Unreal, UX, and physics later.